

НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО
Факультет Программной инженерии и компьютерной техники

Нейроинформатика
Лабораторная работа №1

Выполнил студент
Егорова Варвара Александровна
Группа Р3323
Преподаватель: Русак Алена Викторовна

Содержание	
Задание.....	3
Ход работы.....	4
Ход эксперимента.....	4
Результаты эксперимента.....	16
Исходный код.....	18
Вывод.....	18

Задание

I. Провести экспериментальные исследования с гиперпараметрами модели нейронной сети, чтобы получить наилучший результат на тестовом наборе данных.

1. Попробуйте использовать разное количество нейронов на входном слое: 400, 600, 800, 1200.

2. Добавьте в нейронную сеть скрытый слой с разным количеством нейронов: 200, 300, 400, 600, 800.

3. Добавьте несколько скрытых слоев в сеть с разным количеством нейронов в каждом слое.

4. Используйте разное количество эпох: 10, 15, 20, 25, 30.

5. Используйте разные размеры мини-выборки (batch_size): 10, 50, 100, 200, 500.

6. Также можно провести эксперименты с подбором скорости обучения (learning rate) и с различными оптимизаторами (RMSprop, SGD).

При проведении экспериментов необходимо следить за возможным переобучением.

II. Подготовить отчет с результатами экспериментов и выводами по оптимальной настройке модели.

Ход работы

Для работы использовался датасет `fashion_mnist`, содержащий картинки одежды 28×28 пикселей. В качестве архитектуры модели использовалась полноценная нейронная сеть с функцией активации RELU на скрытых слоях и Softmax на выходном слое.

Для каждого гиперпараметра (количество нейронов на входном слое, количество скрытых слоёв, количество эпох, размер батчей, оптимизатор) была проведена серия экспериментов: один параметр изменялся, остальные были фиксированными.

Ход эксперимента

Было проведено 5 групп экспериментов при следующих изменениях параметров:

- Разное количество нейронов на входном слое: 400, 600, 800, 1200;
- Разное количество скрытых слоёв и разным количеством нейронов на них: 800; 800, 200; 800, 300, 200; 800, 400, 300, 200; 800, 600, 400, 300, 200;
- Разное количество эпох: 10, 15, 20, 25, 30;
- Разные размеры батчей: 10, 50, 100, 200, 500;
- Разные оптимизаторы: Adam, SGD, RMSprop.

Ниже в Листинге 1 представлена загрузка и обработка датасета.

Листинг 1 - Загрузка и обработка датасета

```
(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
classes = ['футболка', 'брюки', 'свитер', 'платье', 'пальто',
           'туфли', 'рубашка', 'кроссовки', 'сумка', 'ботинки']
x_train = x_train.reshape(60000, 784)
x_test = x_test.reshape(10000, 784)
x_train = x_train / 255
x_test = x_test / 255
y_train = utils.to_categorical(y_train, 10)
y_test = utils.to_categorical(y_test, 10)
```

Далее в Листингах 2-7 представлены экспериментальные группы,

функции для обучения моделей и их анализа, код эксперимента, результаты работы программ (результаты обучения и анализа).

Листинг 2 - Экспериментальные группы

```
model_groups = {
    'neurons_layers_comparation' : {
        'base' : {'batch_size' : 200, 'epochs' : 20, 'optimizer' :
'Adam'},
        'var' : {
            'neurons_layer_400' : {'layers' : [400]},
            'neurons_layer_600' : {'layers' : [600]},
            'neurons_layer_800' : {'layers' : [800]},
            'neurons_layer_1200' : {'layers' : [1200]}
        }
    },

    'hidden_layers_comparation' : {
        'base' : {'batch_size' : 200, 'epochs' : 20, 'optimizer' :
'Adam'},
        'var' : {
            '1_layer' : {'layers' : [800]},
            '2_layers' : {'layers' : [800, 200]},
            '3_layers' : {'layers' : [800, 300, 200]},
            '4_layers' : {'layers' : [800, 400, 300, 200]},
            '5_layers' : {'layers' : [800, 600, 400, 300, 200]}
        }
    },

    'epochs_comparation' : {
        'base' : {'batch_size' : 200, 'layers' : [800],
'optimizer' : 'Adam'},
        'var' : {
            '10_epochs' : {'epochs' : 10},
            '15_epochs' : {'epochs' : 15},
            '20_epochs' : {'epochs' : 20},
            '25_epochs' : {'epochs' : 25},
            '30_epochs' : {'epochs' : 30}
        }
    },

    'batch_size_comparation' : {
```

```

        'base' : {'epochs' : 20, 'layers' : [800], 'optimizer' :
'Adam'},
        'var' : {
            'batch_10' : {'batch_size' : 10},
            'batch_50' : {'batch_size' : 50},
            'batch_100' : {'batch_size' : 100},
            'batch_200' : {'batch_size' : 200},
            'batch_500' : {'batch_size' : 500}
        }
    },

    'optimizers_comparation' : {
        'base' : {'batch_size' : 200, 'epochs' : 20, 'layers' :
[800]},
        'var' : {
            'Adam' : {'optimizer' : 'Adam'},
            'SGD' : {'optimizer' : 'SGD'},
            'RMSprop' : {'optimizer' : 'RMSprop'}
        }
    }
}

```

Листинг 3 - Функция создания и обучения модели по заданным параметрам

```

def create_and_train(name, X_train, y_train, layers=[800],
batch_size=200, epochs=20, activator="relu", optimizer="Adam",
verbose=1):
    if verbose:
        print(f"Creating model: {name}")

    model = Sequential()
    model.add(Dense(layers[0], input_dim=784, activation=activator))

    for neurons in layers[1:]:
        model.add(Dense(neurons, activation=activator))

    model.add(Dense(10, activation="softmax"))

    model.compile(loss="categorical_crossentropy",
optimizer=optimizer, metrics=["accuracy"])

```

```

if verbose:
    print(f"Learning model: {name}")

    history = model.fit(X_train, y_train, batch_size=batch_size,
                        epochs=epochs, validation_split=0.2, verbose=0)

    if verbose:
        print(f"Model {name} learned")

    return history.history, model

```

Листинг 4 - Код эксперимента

```

results = {}

for model_group_name, model_group_params in model_groups.items():
    print("="*100)
    print(f'Experimetnal group : {model_group_name}')
    print("="*100 + '\n')

    group_result = []
    base_params = model_group_params['base']

    for var_name, var_params in model_group_params['var'].items():
        params = {**base_params, **var_params}
        case_name = f'{model_group_name} - {var_name}'
        print('Started case: ' + case_name)
        print(f'Params: {params}')

        history, model = create_and_train(
            name=case_name,
            X_train=x_train,
            y_train=y_train,
            layers=params['layers'],
            batch_size=params['batch_size'],
            epochs=params['epochs'],
            optimizer=params['optimizer']
        )

        result = {
            'name': case_name,
            'history': history,

```

```

        'model': model,
        'params': params,
        'group': model_group_name
    }

    group_result.append(result)

    final_val_acc = history['val_accuracy'][-1]
    final_train_acc = history['accuracy'][-1]
    overfitting = final_train_acc - final_val_acc

    print(f"Fast rate: val_acc={final_val_acc:.4f},
overfitting={overfitting:.4f}\n\n")

    results[model_group_name] = group_result

print("="*100)
print('All experiments ended!')
print("="*100)

```

Листинг 5 - Результат обучения

```

=====
Experimetnal group : neurons_layers_comparation
=====

Started case: neurons_layers_comparation - neurons_layer_400
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [400]}
Creating model: neurons_layers_comparation - neurons_layer_400
Learning model: neurons_layers_comparation - neurons_layer_400
Model neurons_layers_comparation - neurons_layer_400 learned
Fast rate: val_acc=0.8974, overfitting=0.0384

Started case: neurons_layers_comparation - neurons_layer_600
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [600]}
Creating model: neurons_layers_comparation - neurons_layer_600
Learning model: neurons_layers_comparation - neurons_layer_600
Model neurons_layers_comparation - neurons_layer_600 learned
Fast rate: val_acc=0.8942, overfitting=0.0453

```



```
Started case: neurons_layers_comparation - neurons_layer_800
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [800]}
Creating model: neurons_layers_comparation - neurons_layer_800
Learning model: neurons_layers_comparation - neurons_layer_800
Model neurons_layers_comparation - neurons_layer_800 learned
Fast rate: val_acc=0.8970, overfitting=0.0433
```

```
Started case: neurons_layers_comparation - neurons_layer_1200
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [1200]}
Creating model: neurons_layers_comparation - neurons_layer_1200
Learning model: neurons_layers_comparation - neurons_layer_1200
Model neurons_layers_comparation - neurons_layer_1200 learned
Fast rate: val_acc=0.8983, overfitting=0.0464
```

```
=====
Experimetnal group : hidden_layers_comparation
=====
```

```
Started case: hidden_layers_comparation - 1_layer
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [800]}
Creating model: hidden_layers_comparation - 1_layer
Learning model: hidden_layers_comparation - 1_layer
Model hidden_layers_comparation - 1_layer learned
Fast rate: val_acc=0.8955, overfitting=0.0465
```

```
Started case: hidden_layers_comparation - 2_layers
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [800, 200]}
Creating model: hidden_layers_comparation - 2_layers
Learning model: hidden_layers_comparation - 2_layers
Model hidden_layers_comparation - 2_layers learned
Fast rate: val_acc=0.8930, overfitting=0.0518
```

```
Started case: hidden_layers_comparation - 3_layers
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [800, 300, 200]}
Creating model: hidden_layers_comparation - 3_layers
Learning model: hidden_layers_comparation - 3_layers
Model hidden_layers_comparation - 3_layers learned
Fast rate: val_acc=0.8957, overfitting=0.0457
```

```
Started case: hidden_layers_comparation - 4_layers
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [800, 400, 300, 200]}
Creating model: hidden_layers_comparation - 4_layers
Learning model: hidden_layers_comparation - 4_layers
Model hidden_layers_comparation - 4_layers learned
Fast rate: val_acc=0.8954, overfitting=0.0479
```

```
Started case: hidden_layers_comparation - 5_layers
Params: {'batch_size': 200, 'epochs': 20, 'optimizer': 'Adam',
'layers': [800, 600, 400, 300, 200]}
Creating model: hidden_layers_comparation - 5_layers
Learning model: hidden_layers_comparation - 5_layers
Model hidden_layers_comparation - 5_layers learned
Fast rate: val_acc=0.8966, overfitting=0.0433
```

```
=====
Experimetnal group : epochs_comparation
=====
```

```
Started case: epochs_comparation - 10_epochs
Params: {'batch_size': 200, 'layers': [800], 'optimizer': 'Adam',
'epochs': 10}
Creating model: epochs_comparation - 10_epochs
Learning model: epochs_comparation - 10_epochs
Model epochs_comparation - 10_epochs learned
Fast rate: val_acc=0.8913, overfitting=0.0230
```

```
Started case: epochs_comparation - 15_epochs
Params: {'batch_size': 200, 'layers': [800], 'optimizer': 'Adam',
```

```

'epochs': 15}
Creating model: epochs_comparation - 15_epochs
Learning model: epochs_comparation - 15_epochs
Model epochs_comparation - 15_epochs learned
Fast rate: val_acc=0.8981, overfitting=0.0324

Started case: epochs_comparation - 20_epochs
Params: {'batch_size': 200, 'layers': [800], 'optimizer': 'Adam',
'epochs': 20}
Creating model: epochs_comparation - 20_epochs
Learning model: epochs_comparation - 20_epochs
Model epochs_comparation - 20_epochs learned
Fast rate: val_acc=0.8963, overfitting=0.0455

Started case: epochs_comparation - 25_epochs
Params: {'batch_size': 200, 'layers': [800], 'optimizer': 'Adam',
'epochs': 25}
Creating model: epochs_comparation - 25_epochs
Learning model: epochs_comparation - 25_epochs
Model epochs_comparation - 25_epochs learned
Fast rate: val_acc=0.8998, overfitting=0.0515

Started case: epochs_comparation - 30_epochs
Params: {'batch_size': 200, 'layers': [800], 'optimizer': 'Adam',
'epochs': 30}
Creating model: epochs_comparation - 30_epochs
Learning model: epochs_comparation - 30_epochs
Model epochs_comparation - 30_epochs learned
Fast rate: val_acc=0.8956, overfitting=0.0641

=====
Experimetnal group : batch_size_comparation
=====

Started case: batch_size_comparation - batch_10
Params: {'epochs': 20, 'layers': [800], 'optimizer': 'Adam',
'batch_size': 10}
Creating model: batch_size_comparation - batch_10

```

```
Learning model: batch_size_comparation - batch_10
Model batch_size_comparation - batch_10 learned
Fast rate: val_acc=0.8848, overfitting=0.0483

Started case: batch_size_comparation - batch_50
Params: {'epochs': 20, 'layers': [800], 'optimizer': 'Adam',
'batch_size': 50}
Creating model: batch_size_comparation - batch_50
Learning model: batch_size_comparation - batch_50
Model batch_size_comparation - batch_50 learned
Fast rate: val_acc=0.8959, overfitting=0.0473

Started case: batch_size_comparation - batch_100
Params: {'epochs': 20, 'layers': [800], 'optimizer': 'Adam',
'batch_size': 100}
Creating model: batch_size_comparation - batch_100
Learning model: batch_size_comparation - batch_100
Model batch_size_comparation - batch_100 learned
Fast rate: val_acc=0.8993, overfitting=0.0447

Started case: batch_size_comparation - batch_200
Params: {'epochs': 20, 'layers': [800], 'optimizer': 'Adam',
'batch_size': 200}
Creating model: batch_size_comparation - batch_200
Learning model: batch_size_comparation - batch_200
Model batch_size_comparation - batch_200 learned
Fast rate: val_acc=0.8962, overfitting=0.0468

Started case: batch_size_comparation - batch_500
Params: {'epochs': 20, 'layers': [800], 'optimizer': 'Adam',
'batch_size': 500}
Creating model: batch_size_comparation - batch_500
Learning model: batch_size_comparation - batch_500
Model batch_size_comparation - batch_500 learned
Fast rate: val_acc=0.8946, overfitting=0.0412
```

```
=====
```

```

Experimetnal group : optimizers_comparation
=====

Started case: optimizers_comparation - Adam
Params: {'batch_size': 200, 'epochs': 20, 'layers': [800],
'optimizer': 'Adam'}
Creating model: optimizers_comparation - Adam
Learning model: optimizers_comparation - Adam
Model optimizers_comparation - Adam learned
Fast rate: val_acc=0.9014, overfitting=0.0401

Started case: optimizers_comparation - SGD
Params: {'batch_size': 200, 'epochs': 20, 'layers': [800],
'optimizer': 'SGD'}
Creating model: optimizers_comparation - SGD
Learning model: optimizers_comparation - SGD
Model optimizers_comparation - SGD learned
Fast rate: val_acc=0.8453, overfitting=0.0069

Started case: optimizers_comparation - RMSprop
Params: {'batch_size': 200, 'epochs': 20, 'layers': [800],
'optimizer': 'RMSprop'}
Creating model: optimizers_comparation - RMSprop
Learning model: optimizers_comparation - RMSprop
Model optimizers_comparation - RMSprop learned
Fast rate: val_acc=0.8961, overfitting=0.0393

=====
All experiments ended!
=====

```

Листинг 6 - Функция для анализа

```

def analyze_group(name, group_results):
    print('=' * 100)
    print(f'Analyze group: {name}')
    print('=' * 100)

    sorted_results = sorted(group_results, key=lambda x:

```

```

x['history']['val_accuracy'][-1], reverse=True)

print(f"{'Модель':<30} {'Val Acc':<8} {'Train Acc':<10} {'Val Loss':<8} {'Train Loss':<10} {'Overfit':<8} {'Параметры'}")
print("=" * 110)

for result in sorted_results:
    history = result['history']
    params = result['params']

    val_acc = history['val_accuracy'][-1]
    train_acc = history['accuracy'][-1]
    val_loss = history['val_loss'][-1]
    train_loss = history['loss'][-1]
    overfitting = train_acc - val_acc

    if name == 'neurons_layers_comparation':
        param_desc = f"Heuron layer: {params['layers'][0]}"
    elif name == 'hidden_layers_comparation':
        layers_str = '-'.join(map(str, params['layers']))
        param_desc = f"Layers: {layers_str}"
    elif name == 'epochs_comparation':
        param_desc = f"Epochs: {params['epochs']}"
    elif name == 'batch_size_comparation':
        param_desc = f"Batch: {params['batch_size']}"
    elif name == 'optimizer_comparation':
        param_desc = f"Optim: {params['optimizer']}"
    else:
        param_desc = str(params)

    print(f"{name:<30} {val_acc:.4f} {train_acc:.4f} {val_loss:.4f} {train_loss:.4f} {overfitting:.4f} {param_desc}")
    print('-' * 100)

```

Листинг 7 - Результат анализа

```

=====
All groups analisys
=====
Analyze group: neurons_layers_comparation
=====
Модель                Val Acc  Train Acc  Val Loss  Train Loss  Overfit  Параметры
=====
neurons_layers_comparation  0.8983  0.9448    0.3145    0.1525      0.0464  Heuron layer: 1200
=====

```

```

-----
neurons_layers_comparation      0.8974  0.9359   0.3022  0.1703   0.0384  Heuron layer: 400
-----
neurons_layers_comparation      0.8970  0.9403   0.3048  0.1616   0.0433  Heuron layer: 800
-----
neurons_layers_comparation      0.8942  0.9395   0.3137  0.1655   0.0453  Heuron layer: 600
-----
=====
Analyze group: hidden_layers_comparation
=====
Модель                          Val Acc  Train Acc  Val Loss  Train Loss  Overfit  Параметры
=====
hidden_layers_comparation      0.8966  0.9399   0.3582   0.1565   0.0433  Layers:
800-600-400-300-200
-----
hidden_layers_comparation      0.8957  0.9414   0.3568   0.1524   0.0457  Layers: 800-300-200
-----
hidden_layers_comparation      0.8955  0.9420   0.3136   0.1587   0.0465  Layers: 800
-----
hidden_layers_comparation      0.8954  0.9433   0.3448   0.1475   0.0479  Layers:
800-400-300-200
-----
hidden_layers_comparation      0.8930  0.9448   0.3342   0.1441   0.0518  Layers: 800-200
-----
=====
Analyze group: epochs_comparation
=====
Модель                          Val Acc  Train Acc  Val Loss  Train Loss  Overfit  Параметры
=====
epochs_comparation             0.8998  0.9514   0.3137   0.1339   0.0515  Epochs: 25
-----
epochs_comparation             0.8981  0.9305   0.2971   0.1876   0.0324  Epochs: 15
-----
epochs_comparation             0.8963  0.9417   0.3045   0.1572   0.0455  Epochs: 20
-----
epochs_comparation             0.8956  0.9596   0.3464   0.1106   0.0641  Epochs: 30
-----
epochs_comparation             0.8913  0.9143   0.3070   0.2320   0.0230  Epochs: 10
-----
=====
Analyze group: batch_size_comparation
=====
Модель                          Val Acc  Train Acc  Val Loss  Train Loss  Overfit  Параметры
=====
batch_size_comparation         0.8993  0.9440   0.3195   0.1508   0.0447  Batch: 100
-----
batch_size_comparation         0.8962  0.9429   0.3203   0.1569   0.0468  Batch: 200
-----
batch_size_comparation         0.8959  0.9432   0.3260   0.1515   0.0473  Batch: 50
-----
batch_size_comparation         0.8946  0.9358   0.3068   0.1796   0.0412  Batch: 500
-----
batch_size_comparation         0.8848  0.9330   0.4076   0.1788   0.0483  Batch: 10
-----
=====
Analyze group: optimizers_comparation
=====
Модель                          Val Acc  Train Acc  Val Loss  Train Loss  Overfit  Параметры
=====
optimizers_comparation         0.9014  0.9415   0.3032   0.1589   0.0401  {'batch_size': 200,
'epochs': 20, 'layers': [800], 'optimizer': 'Adam'}
-----
optimizers_comparation         0.8961  0.9354   0.3145   0.1735   0.0393  {'batch_size': 200,

```

```
'epochs': 20, 'layers': [800], 'optimizer': 'RMSprop'}
-----
optimizers_comparation      0.8453  0.8521   0.4480  0.4368   0.0069  {'batch_size': 200,
'epochs': 20, 'layers': [800], 'optimizer': 'SGD'}
```

Результаты эксперимента

Далее в таблицах 1-5 представлена интерпретация результата анализа обученных моделей. Параметры, которые указывались в качестве неизменяемых были равны: batch_size = 200, epochs = 20, layers = [800], optimizer = 'Adam'.

Группа экспериментов: количество нейронов на входном слое					
Val accuracy	Train accuracy	Val Loss	Train Loss	Overfit	Количество нейронов
0.8983	0.9448	0.3145	0.1525	0.0464	1200
0.8974	0.9359	0.3022	0.1703	0.0384	400
0.8970	0.9403	0.3048	0.1616	0.0433	800
0.8942	0.9395	0.3137	0.1655	0.0453	600

Таблица 1 - Результаты экспериментов с изменяемым количеством нейронов на входном слое

Группа экспериментов: количество скрытых слоёв и нейронов в них					
Val accuracy	Train accuracy	Val Loss	Train Loss	Overfit	Количество нейронов в скрытых слоях
0.8966	0.9399	0.3582	0.1565	0.0433	800-600-400-300-200
0.8957	0.9414	0.3568	0.1524	0.0457	800-300-200
0.8955	0.9420	0.3136	0.1587	0.0465	800
0.8954	0.9433	0.3448	0.1475	0.0479	800-400-300-200
0.8930	0.9448	0.3342	0.1441	0.0518	800-200

Таблица 2 - Результаты экспериментов с изменяемым количеством скрытых слоёв и нейронов в них

Группа экспериментов: количество эпох					
Val accuracy	Train accuracy	Val Loss	Train Loss	Overfit	Количество эпох
0.8998	0.9514	0.3137	0.1339	0.0515	25
0.8981	0.9305	0.2971	0.1876	0.0324	15
0.8963	0.9417	0.3045	0.1572	0.0455	20
0.8956	0.9596	0.3464	0.1106	0.0641	30
0.8913	0.9143	0.3070	0.2320	0.0230	10

Таблица 3 - Результаты экспериментов с изменяемым количеством эпох

Группа экспериментов: размер батча					
Val accuracy	Train accuracy	Val Loss	Train Loss	Overfit	Размер батча
0.8993	0.9440	0.3195	0.1508	0.0447	100
0.8962	0.9429	0.3203	0.1569	0.0468	200
0.8959	0.9432	0.3260	0.1515	0.0473	50
0.8946	0.9358	0.3068	0.1796	0.0412	500
0.8848	0.9330	0.4076	0.1788	0.0483	10

Таблица 4 - Результаты экспериментов с изменяемым размером батча

Группа экспериментов: оптимизаторы					
Val accuracy	Train accuracy	Val Loss	Train Loss	Overfit	Оптимизатор

0.9014	0.9415	0.3032	0.1589	0.0401	Adam
0.8961	0.9354	0.3145	0.1735	0.0393	RMSprop
0.8453	0.8521	0.4480	0.4368	0.0069	SGD

Таблица 5 - Результаты экспериментов с изменяемым оптимизатором

Исходный код

Исходный полный код программы представлен в Google Colab по ссылке: [NeuroInfLab1](#)

Вывод

По результатам эксперимента, представленных в таблицах 1-5 можно сделать следующие выводы:

1. При изменении количества нейронов на входном слое лучшие результаты показали модели с 1200 и 400 нейронами. Их точность составила 89,83% и 89,74% соответственно при сравнительно низком проценте переобучения: 4,64% и 3,84%. Модели с 600 и 800 нейронами при уровне точности ниже имеют примерно такой же процент переобучения.
2. Модели с разными архитектурами показали примерно одинаковые результаты как по точности, так и по проценту переобучения, за исключением модели с архитектурой '800-200', у которой процент точности самый низкий и самый высокий процент переобучения. Вероятно, это произошло из-за резкого снижения количества нейронов.
3. Модель с 15-ю эпохами показала лучший результат в своей группе: низкий процент переобучения (3,24%) при достаточно высоком уровне точности (89,81%). Другие модели имеют сравнительно низкую точность и высокий процент переобучения.
4. Модель с размером батча 100 имеет наивысшую точность в группе (89,93%) и один из самых низких процентов переобучения (4,47%).

5. Модель с оптимизатором Adam продемонстрировала лучший результат по точности (90,14%) и имеет средний уровень переобучения (4,01%). Похожий результат показал и оптимизатор RMSprop. Можно также обратить внимание на модель с оптимизатором SGD, которая показала наинизший уровень переобучения (0,69%) и при этом самую низкую точность (84,53%) среди всех моделей во всех экспериментах.

Эксперимент показал, что необходимо тщательно подбирать гиперпараметры нейронных сетей. Баланс точности и устойчивости к переобучению достигается при средних параметрах, а не за счёт перегрузки модели.